Spring Framework – Day 11 Notes

Topic: Collection Type Dependency Injection (List, Set, Map, Properties)

---

1. Recap: Injecting One Object Inside Another

Before we look at collections, let us recall how we inject one class object into another class using setter or constructor injection. Class A holds an object of Class B as one of its properties (this is called composition). Spring creates both beans and wires B into A using the <property> tag with autowiring.

```
class A {
    int a;
    int b;
    B b;                 // Object of class B (composition)

    // constructor
    // setter methods
    // toString()
}

class B {
    int c;
    int d;

    // constructor
    // setter methods
    // toString()
}
```

Corresponding application-context.xml:

```
<beans>
    <bean id="a" class="package.classA" autowire="byName">
        <property name="a" value="100"></property>
        <property name="b" value="200"></property>
    </bean>

    <bean id="b" class="package.classB">
        <property name="c" value="300"></property>
    </bean>
</beans>
```

In the same way we injected object B into class A, the question arises: can we also inject a collection, such as an **ArrayList**, in a similar manner?

```
class A {
   int a;
   int b;
   List<String> data;
}
```

If we try to configure it the same way we configured bean B, we run into a problem:

```
<bean id="data" class="java.util.ArrayList">

    <!-- Problem: We cannot use setter or constructor
         injection here because we do not have the source
         code of the ArrayList class. We cannot add our own
         constructor or setter to a JDK class. -->
```

```
</bean>
```

Since we cannot modify the ArrayList class to add setters or constructors, Spring provides **separate XML tags** to inject each type of collection dependency directly, without needing any setter or constructor inside the collection class itself.

2. Injecting a List

To inject a **List** type property, Spring provides the <list> tag inside <property>. Below is a Student class that holds a List of String values.

```
package com.day11;

import java.util.List;

public class Student {
    public List<String> data;

    public Student() {
        // default constructor
    }

    public List<String> getData() {
        return data;
    }

    public void setData(List<String> data) {
        this.data = data;
    }

    @Override
    public String toString() {
        return "Student [data=" + data + "]";
    }
}
```

application-context.xml for the Student bean:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="student" class="com.day11.Student">
        <property name="data">
            <list>
                <value>Prashant</value>
                <value>Tanjiro</value>
                <value>Sachin</value>
            </list>
        </property>
    </bean>
</beans>
```

3. Injecting a Set

To inject a **Set** type property, Spring provides the <set> tag. A Set does not allow duplicate values. Below is a Course class that holds a Set of course names.

```
package com.day11;
```

```
import java.util.Set;

public class Course {
    private Set<String> courseName;

    public Course() {
        // default constructor
    }

    public Course(Set<String> courseName) {
        super();
        this.courseName = courseName;
    }

    public Set<String> getCourseName() {
        return courseName;
    }

    public void setCourseName(Set<String> courseName) {
        this.courseName = courseName;
    }

    @Override
    public String toString() {
        return "Course [courseName=" + courseName + "]";
    }
}
```

application-context.xml for the Course bean:

```
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="course" class="com.day11.Course">
        <property name="courseName">
            <set>
                <value>JAVA</value>
                <value>Python</value>
                <value>Spring Framework</value>
            </set>
        </property>
    </bean>
</beans>
```

4. Injecting a Map

To inject a **Map** type property, Spring provides the <map> tag with <entry> elements for each key-value pair. Below is a Duration class that holds a Map of subject name to number of hours.

```
package com.day11;

import java.util.Map;

public class Duration {
    private Map<String, Integer> value;

    public Duration() {
        // default constructor
    }

    public Duration(Map<String, Integer> value) {
```

```
        super();
        this.value = value;
    }

    public Map<String, Integer> getValue() {
        return value;
    }

    public void setValue(Map<String, Integer> value) {
        this.value = value;
    }

    @Override
    public String toString() {
        return "Duration [value=" + value + "]";
    }
}
```

application-context.xml for the Duration bean:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="duration" class="com.day11.Duration">
        <property name="value">
            <map>
                <entry key="java" value="3"></entry>
                <entry key="python" value="2"></entry>
                <entry key="GenAI" value="2"></entry>
            </map>
        </property>
    </bean>
</beans>
```

5. Collections Holding Bean References (Object Type)

A collection does not always have to hold simple values like String or Integer. It can also hold references to other beans. The example below shows a CourseDetail class where:

- A **List<Duration>** holds a list of Duration bean references.
- A **Map<Student, Course>** holds Student beans as keys and Course beans as values.

```
package com.day11;

import java.util.List;
import java.util.Map;

public class CourseDetail {

    private List<Duration> duration;
    private Map<Student, Course> detail;

    public CourseDetail() {
        // default constructor
    }

    public CourseDetail(List<Duration> duration, Map<Student, Course> detail) {
        super();
        this.duration = duration;
        this.detail = detail;
```

```
    }

    public List<Duration> getDuration() {
        return duration;
    }

    public void setDuration(List<Duration> duration) {
        this.duration = duration;
    }

    public Map<Student, Course> getDetail() {
        return detail;
    }

    public void setDetail(Map<Student, Course> detail) {
        this.detail = detail;
    }

    @Override
    public String toString() {
        return "CourseDetail [duration=" + duration + ", detail=" + detail + "]";
    }
}
```

Complete application-context.xml combining Student, Course, Duration, and CourseDetail beans. Notice the use of **<ref bean="...">** to place an existing bean inside a list, and **key-ref** / **value-ref** to place existing beans as a key and value inside a map entry:

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="student" class="com.day11.Student">
        <property name="data">
            <list>
                <value>Prashant</value>
                <value>Tanjiro</value>
                <value>Sachin</value>
            </list>
        </property>
    </bean>

    <bean id="course" class="com.day11.Course">
        <property name="courseName">
            <set>
                <value>JAVA</value>
                <value>Python</value>
                <value>Spring Framework</value>
            </set>
        </property>
    </bean>

    <bean id="duration" class="com.day11.Duration">
        <property name="value">
            <map>
                <entry key="java" value="3"></entry>
                <entry key="python" value="2"></entry>
                <entry key="GenAI" value="2"></entry>
            </map>
        </property>
    </bean>
```

```
    <bean id="courseDetail" class="com.day11.CourseDetail">

        <property name="duration">
            <list>
                <ref bean="duration"/>
            </list>
        </property>

        <property name="detail">
            <map>
                <entry key-ref="student" value-ref="course"></entry>
            </map>
        </property>
    </bean>

</beans>
```

6. Map vs Properties

When to use Map?

Use Map when the values can be of any object type, such as String, Integer, or even another bean.

```
Map<String, Integer> marks;
Map<Student, Course> details;
Map<String, Employee> employees;
```

When to use Properties?

Use Properties when you are storing configuration or settings, where both the key and the value are always of type String.

```
package com.day11;

import java.util.Properties;

public class CompanyResource {
    private Properties database;

    public CompanyResource() {
        // default constructor
    }

    public CompanyResource(Properties database) {
        super();
        this.database = database;
    }

    public Properties getDatabase() {
        return database;
    }

    public void setDatabase(Properties database) {
        this.database = database;
    }

    @Override
    public String toString() {
        return "CompanyResource [database=" + database + "]";
    }
}
```

application-context.xml using the <props> tag:

```xml
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="sources" class="com.day11.CompanyResource">
        <property name="database">
            <props>
                <prop key="username">system</prop>
                <prop key="password">manager</prop>
                <prop key="driverName">oracle.jdbc.driver.OracleDriver</prop>
                <prop key="url">jdbc:oracle:thin:@localhost:1521:xe</prop>
            </props>
        </property>
    </bean>

</beans>
```

7. Quick Summary

Collection Type	XML Tag	Allows Duplicates?	Typical Use
List	<list>	Yes	Ordered collection of values or bean refs
Set	<set>	No	Unique collection of values or bean refs
Map	<map> / <entry>	Keys unique	Key-value pairs of any object type
Properties	<props> / <prop>	Keys unique	Key-value pairs where both are Strings (configuration/settings)

Key takeaway: A collection class such as ArrayList, HashSet, or HashMap does not have source code that we can modify to add a constructor or setter. So Spring gives dedicated XML tags (<list>, <set>, <map>, <props>) to inject these collection type dependencies directly into our own class properties.

Practice MCQs – Test Your Understanding

Attempt all questions below without looking back at the notes. Answers are intentionally not given here so you can self-check your understanding first.

1. Look at this Student class field and its XML wiring. What is wrong with the XML?

```
public List<String> data;

<bean id="student" class="com.day11.Student">
    <property name="data">
        <set>
            <value>Prashant</value>
            <value>Tanjiro</value>
        </set>
    </property>
</bean>
```

A. The property name should be "Data" with a capital D

B. The field is declared as List but the XML wires it using , which is the wrong tag for a List field

C. tags cannot be used inside

D. Nothing is wrong; List and Set tags are interchangeable in Spring

2. A Course class stores course names in a Set. Which XML snippet correctly injects three course names, and why would the alternative be a mistake?

```
private Set<String> courseName;
```

A. JAVA -- correct, because List and Set both accept duplicates

B. JAVAPython -- correct, matches the Set field type

C. -- correct, Map can replace Set

D. Python -- correct, Properties works for any type

3. Find the mistake in this Map injection for the Duration class field private Map value;

```
<bean id="duration" class="com.day11.Duration">
    <property name="value">
        <map>
            <entry key="java">3</entry>
            <entry key="python" value="2"></entry>
        </map>
    </property>
</bean>
```

A. The first is missing the value attribute; it should be

B. The tag should be for multiple entries

C. The property name should be "values" instead of "value"

D. There is no mistake; both entry formats shown are valid in Spring XML

4. A CourseDetail bean needs an existing 'duration' bean placed inside its List property. Which line correctly does this?

```
private List<Duration> duration;
```

A.

B.

C.

D.

5. In the CourseDetail bean, the Map detail property must use existing 'student' and 'course' beans as key and value. Which entry is correct?

```
private Map<Student, Course> detail;
```

A.

B.

C.

D.

6. A student wired database configuration using this XML. What is the mistake, given the field is private Properties database;?

```
<bean id="sources" class="com.day11.CompanyResource">
    <property name="database">
        <map>
            <entry key="username" value="system"></entry>
            <entry key="password" value="manager"></entry>
        </map>
    </property>
</bean>
```

A. The bean id "sources" is not allowed for Properties beans

B. Properties fields should be wired using with value, not with

C. The password value should never be a String

D. There is no mistake; and produce the same result for a Properties field

7. What will happen if this XML is used to wire a field declared as private Set courseName;?

```
<property name="courseName">
    <set>
        <value>JAVA</value>
        <value>JAVA</value>
        <value>Python</value>
    </set>
</property>
```

A. Spring will throw an exception because of the duplicate "JAVA" value

B. The resulting Set will silently drop the duplicate and contain only "JAVA" and "Python"

C. The resulting Set will contain "JAVA" twice and "Python" once

D. Spring will convert the into a List automatically

8. Identify the correct syntax for a Properties field named database, given one entry is missing its closing tag below.

```
<property name="database">
    <props>
        <prop key="username">system</prop>
        <prop key="password">manager
        <prop key="url">jdbc:oracle:thin:@localhost:1521:xe</prop>
    </props>
</property>
```

A. The XML is valid as written; unclosed tags are allowed for prop elements

B. The manager line is missing its closing tag

C. The prop key should be an attribute called "name" instead of "key"

D. Properties do not support a tag at all; only can be used

9. Given class CourseDetail(List duration, Map detail) has only a constructor and no setters, which injection style must the XML use?

```
public CourseDetail(List<Duration> duration, Map<Student, Course> detail) {
    this.duration = duration;
    this.detail = detail;
}
```

A. tags only, since constructor injection cannot wire collections

B. tags, since only a parameterized constructor is available

C. Either or , Spring picks automatically at runtime

D. Field injection using @Autowired only

10. Why is it not possible to add a custom setter directly to java.util.ArrayList to enable normal setter injection, and what does Spring provide instead?

A. ArrayList is a final class, so Spring provides the @Final annotation to bypass it

B. We do not have the ArrayList source code to modify, so Spring provides ready-made tags like , , , and to inject collection values directly

C. ArrayList already has a setter, so no special tag is needed

D. Spring requires every collection class to be rewritten as a custom bean before injection

Tip: Once you have selected your answers, revisit the corresponding section in these notes to verify. If you got most of them right, you have a solid grasp of collection type dependency injection in Spring!